

Reg.No : 192525047

Name : G.Karthik Reddy

CSA0609 - DESIGN AND ANALYSIS OF ALGORITHM

Experiment 7: Selection Sort on Random, Reverse and Sorted Arrays

Aim

To implement Selection Sort and observe its behavior (comparisons and swaps) on random, reverse-ordered and already-sorted arrays.

Algorithm

1. Start with $i = 0$.
2. Assume the element at position i is the minimum.
3. Compare it with every element to its right and note the position of the smallest one.
4. Swap the smallest element with the element at position i .
5. Increase i by 1 and repeat steps 2 to 4 until $i = n-2$.
6. Repeat the whole process for the random, reverse and sorted arrays and compare the counts.

Pseudo code

SelectionSort(A, n)

for $i = 0$ to $n-2$

$\text{min} = i$

 for $j = i+1$ to $n-1$

$\text{comparisons}++$

 if $A[j] < A[\text{min}]$

$\text{min} = j$

 if $\text{min} \neq i$

 swap $A[i], A[\text{min}]$

$\text{swaps}++$

Source code (C)

c

#include <stdio.h>

```

void selectionSort(int a[], int n) {
    int i, j, min, temp, comp = 0, swaps = 0;
    for (i = 0; i < n - 1; i++) {
        min = i;
        for (j = i + 1; j < n; j++) {
            comp++;
            if (a[j] < a[min])
                min = j;
        }
        if (min != i) {
            temp = a[i];
            a[i] = a[min];
            a[min] = temp;
            swaps++;
        }
    }
    printf("Sorted   : ");
    for (i = 0; i < n; i++) printf("%d ", a[i]);
    printf("\nComparisons = %d, Swaps = %d\n\n", comp, swaps);
}

int main() {
    int random[] = {29, 10, 14, 37, 13, 5, 42, 20};
    int reverse[] = {80, 70, 60, 50, 40, 30, 20, 10};
    int sorted[] = {10, 20, 30, 40, 50, 60, 70, 80};
    int n = 8;
    printf("Random Array\n");
    selectionSort(random, n);
    printf("Reverse Array\n");
    selectionSort(reverse, n);
}

```

```

printf("Sorted Array\n");

selectionSort(sorted, n);

return 0;

}

```

Output

Random Array

Sorted : 5 10 13 14 20 29 37 42

Comparisons = 28, Swaps = 5

Reverse Array

Sorted : 10 20 30 40 50 60 70 80

Comparisons = 28, Swaps = 4

Sorted Array

Sorted : 10 20 30 40 50 60 70 80

Comparisons = 28, Swaps = 0

The screenshot shows a C++ IDE with the following code in `DAA.c`:

```

1 #include <stdio.h>
2
3 void selectionSort(int a[], int n) {
4     int i, j, min, temp, comp = 0, swaps = 0;
5     for (i = 0; i < n - 1; i++) {
6         min = i;
7         for (j = i + 1; j < n; j++) {
8             comp++;
9             if (a[j] < a[min])
10                min = j;
11         }
12         if (min != i) {
13             temp = a[i];
14             a[i] = a[min];
15             a[min] = temp;
16             swaps++;
17         }
18     }
19     printf("Sorted : ");
20     for (i = 0; i < n; i++) printf("%d ", a[i]);
21     printf("\nComparisons = %d, Swaps = %d\n", comp, swaps);
22 }
23
24 int main() {
25     int random[] = {29, 10, 14, 37, 13, 5, 42, 20};
26     int reverse[] = {80, 70, 60, 50, 40, 30, 20, 10};
27     int sorted[] = {10, 20, 30, 40, 50, 60, 70, 80};
28     int n = 8;
29
30     printf("Random Array\n");
31     selectionSort(random, n);
32 }

```

The output window shows the following results:

```

Random Array
Sorted : 5 10 13 14 20 29 37 42
Comparisons = 28, Swaps = 5

Reverse Array
Sorted : 10 20 30 40 50 60 70 80
Comparisons = 28, Swaps = 4

Sorted Array
Sorted : 10 20 30 40 50 60 70 80
Comparisons = 28, Swaps = 0

-----
Process exited after 0.8998 seconds with return value 0
Press any key to continue . . .

```

The compilation results window shows:

```

Compilation results...
-----
- Errors: 0
- Warnings: 0
- Output Filename: C:\Users\karth\OneDrive\Documents\DAA.exe
- Output Size: 129.6318359375 KiB
- Compilation Time: 0.44s

```

Result

Selection Sort was implemented and tested on random, reverse and sorted arrays. The number of comparisons is always $n(n-1)/2$ (28 for $n = 8$), whatever the input order, so the time complexity is $O(n^2)$ in the best, average and worst cases. Only the number of swaps changes with the input.